

Отказоустойчивая университетская информационная система

Fault Tolerant University Information System

Технический отчёт по midterm | Fault Tolerance and Dependable Computing

Campus reliability lab | 5 октября 2026 года | Русская версия

Студент: Tastan Magzhan

Группа: CSE-2505M

Преподаватель: Azamat Serek

В компактной работающей университетской системе сравниваются намеренно упрощённая baseline и конфигурация с fault tolerance (FT). Три бизнес-сервиса и общий сервис хранения SQLite позволяют исследовать сбои, recovery и согласованность транзакций в реальных HTTP-экспериментах.

Набор данных содержит 54 контролируемых запуска: девять сценариев, две конфигурации и три повторения. Измерения проведены на локальных процессах одного компьютера с Windows. Отказ физического узла моделируется остановкой группы процессов; испытания на нескольких физических серверах не проводились.

Репозиторий: https://github.com/zzmaga/fttr_midterm

Данные: results/final | Запуск: README.md | Демонстрация: docs/DEMO.md

1 Введение и мотивация проекта

Университетская платформа не должна повторно учитывать оплату обучения при потере ответа. Доступ к платежам также не должен зависеть от необязательного сервиса transcript. Проект демонстрирует availability и integrity. Небольшая архитектура позволяет проследить каждый механизм защиты в коде и показать его на короткой устной защите.

Основной результат - fault containment: второй процесс payment сохраняет доступ к платежам при отказе первой реплики, а транзакции и idempotency предотвращают частичное или повторное применение платежа. Отказ общей базы данных по-прежнему прерывает зависящие от неё операции. Redundancy помогает только в пределах реально защищённой области отказов.

2 Требования и допущения

Задание требует baseline, не менее трёх независимых сервисов, пяти реалистичных сценариев отказа, двух инфраструктурных и четырёх программных механизмов защиты, контролируемого сравнения до и после их внедрения, расчётов reliability и демонстрации трёх сценариев recovery. Реализованы все шесть явно перечисленных в задании сценариев и три дополнительных эксперимента по integrity и recovery.

Критерии учебного стенда: отсутствие неуспешных запросов списка платежей при падении одной payment-реплики в FT; отсутствие расхождений между балансом и ledger и повторяющихся ключей во всех FT-испытаниях; восстановление полноценных ответов после устранения сбоя; наличие записей об обнаружении, восстановлении и запросах. Эти критерии относятся к экспериментам и не являются production SLA.

У двух вымышленных студентов начальная задолженность составляет по 100000 тийн. Суммы хранятся целыми числами: 100 тийн = 1 KZT. До измерений каждого эксперимента записывается начальный платёж в 100 тийн, поэтому сохранность проверяется на непустых данных. Управление fault injection доступно локально по token. Аутентификация пользователей, интеграция с банком и развёртывание на нескольких хостах не входят в объём проекта.

3 Dependability и fault model

Dependability включает availability, integrity и recoverability. Reliability характеризует непрерывную работу за интервал; availability допускает восстановление и учитывает долю доступного времени. Fault - причина сбоя; error - некорректное внутреннее состояние или результат; failure - нарушение обещанного поведения сервиса.

Пример: завершение payment1 является fault; отказ соединения в gateway - error; неуспешный клиентский запрос платежа - service failure. В FT gateway может обратиться к payment2 и скрыть ошибку от клиента. Потеря подтверждения представляет собой omission fault: платёж уже может существовать, хотя клиент увидел отказ.

Модель включает crash-stop, временные задержки и ошибки, потерю ответа после commit и прерывание операции. Совместная остановка процессов моделирует отказ узла до явного восстановления. Byzantine failures, произвольное повреждение диска, потеря всего хоста и вредоносные запросы исключены. Degraded transcript полезен, но не считается полноценным ответом при расчёте availability.

4 Reliability и анализ отказов

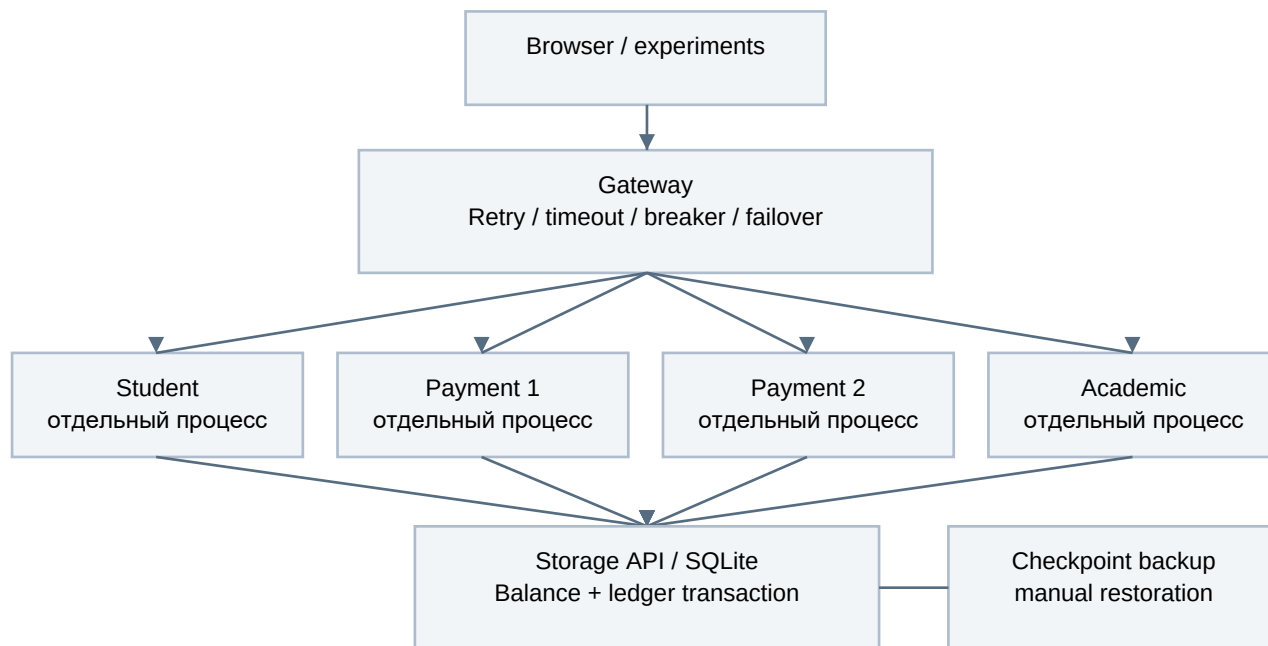
Качественный failure analysis связывает каждый внедряемый fault с внутренней ошибкой, последствиями для клиента и оставшимися рисками. Вероятности возникновения и численные оценки риска без экспериментального обоснования не назначаются.

Fault	Error и последствия	Защита и ограничения
Application crash	Payment endpoint не может соединиться с payment1.	Реплика и failover сохраняют доступ; gateway и база остаются single points of failure.
Database outage	Нет соединения со storage; бизнес-API не могут читать и писать.	Ограниченный retry и контролируемый 503; restart восстанавливает доступ. Реплики базы нет.
Service timeout	Ответ academic превышает downstream timeout.	Circuit breaker ограничивает вызовы; возвращается помеченный cache. Полный transcript недоступен.
Node A simulation	Payment1 и academic завершаются вместе.	Payment2 и student продолжают работу. Потеря всего хоста не покрыта.
Interrupted payment	Баланс изменён до вставки записи в ledger.	FT выполняет rollback; baseline оставляет расхождение.
Concurrent load	Очереди и конкуренция за общее storage увеличивают latency.	Два worker распределяют запросы; общая SQLite выполняет запись последовательно.
Duplicate / lost reply	Один запрос платежа доставляется несколько раз.	Общий unique key и проверка payload возвращают первоначальный платёж.
Checkpoint restore	Восстанавливается более раннее состояние базы.	Бэкап возвращает согласованное состояние, но теряет записи после checkpoint.

Упрощённое fault tree недоступности платежей: отказ gateway OR отказ storage OR (недоступность payment1 AND недоступность payment2) OR общий отказ хоста/сети. В baseline достаточно отказа единственного payment-процесса. Причины потери integrity рассматриваются отдельно: отсутствие atomicity, отсутствие idempotency или возврат к старому checkpoint без reconciliation.

При иллюстративном допущении независимости две реплики с availability $A=0.99$ каждая дают $1-(1-A)^2=0.9999$. Если availability gateway и storage также равна 0.99, последовательная система даёт $0.99 \times 0.99 \times 0.9999 = 0.98000199$. Это условные числа, а не экспериментальные оценки. Общий хост дополнительно нарушает допущение независимости.

5 Архитектура системы



Сервисы пишут JSON logs; стенд сохраняет запросы, faults и результаты audit.

Рисунок 1. Каждый бизнес-сервис работает в отдельном HTTP-процессе. В FT добавляется payment2. Gateway реализован небольшим приложением FastAPI, поэтому выбор маршрута и состояние circuit breaker легко проверить в коде. Mermaid-схема находится в docs/architecture.md.

Порты: gateway - 8100, student - 8101, payment - 8102 и 8105, academic - 8103, storage - 8104. Эксперименты используют 8200-8205, тесты - 8400-8405. Launcher проверяет доступность портов и управляет только собственными деревьями дочерних процессов.

Все бизнес-сервисы обращаются к одному storage API. Только его процесс открывает SQLite. Единая точка обработки транзакций обеспечивает согласованную проверку ключей между репликами. В gateway хранятся отдельные circuit breaker для endpoint и transcript cache в памяти. Потеря cache после restart допустима: сервис явно возвращает degraded-ответ без сохранённого transcript.

В моделируемый node A входят payment1 и academic; в node B - payment2 и student. Это логические группы на одном компьютере. Каждый процесс пишет JSON logs, а экспериментальный стенд отдельно сохраняет времена запросов и снимки состояния.

6 Проектирование hardware fault tolerance

Инфраструктурная защита включает дополнительные payment instances, round-robin routing с failover и восстановление backup. Gateway обходит завершённый payment-процесс. Позднее стенд перезапускает его по сценарию эксперимента; это запланированный restart, а не автономное восстановление production orchestrator.

SQLite backup создаёт согласованный checkpoint-файл [2]. В restore-эксперименте возврат к checkpoint намеренно удаляет корректный более поздний платёж, наглядно показывая recovery-point loss. Это логическое восстановление на одном диске. В production backup нужно хранить в другой области отказа и согласовывать с последующими транзакциями через reconciliation.

RAID использует дисковую redundancy для защиты от определённых отказов дисков, но не отменяет логически неверную запись и не заменяет backup. ECC memory обнаруживает и исправляет поддерживаемые ошибки битов памяти, но не исправляет алгоритм приложения. RAID и ECC описаны теоретически; их реализация и измерение в проекте не заявляются.

7 Проектирование software fault tolerance

Механизм	Реализация и границы применения
Retry и timeout	Gateway выполняет максимум 3 попытки с backoff 50/100 ms и HTTPX timeout 300 ms. В baseline одна попытка и защитный timeout 1 s [3].
Circuit breaker	После двух downstream failures цепь для endpoint открывается. Через 1 s одна HALF_OPEN probe может закрыть её или открыть повторно [5].
Idempotency	Unique key SQLite проверяется внутри BEGIN IMMEDIATE. Тот же key и payload возвращают прежнюю запись; другой payload получает 409.
Checkpoint и rollback	До изменения баланса создаётся savepoint; interruption вызывает rollback. Общая транзакция выполняет commit баланса и ledger вместе [1].
Health и degradation	Liveness endpoints показывают состояние процессов. При отказе academic возвращается явно помеченный cache; платёж не получает фиктивный success.

8 Реализация

Python, FastAPI, HTTPX и SQLite позволяют сохранить небольшой объём приложения. app/service.py запускается независимо для трёх ролей. app/database.py управляет storage и audit; app/resilience.py - политикой recovery. В baseline намеренно нет idempotency, а commit баланса выполняется до вставки платежа. Проверка входных данных сохранена в обеих версиях.

Автоматические тесты проверяют concurrent duplicates между репликами, конфликт payload, потерю ответа после commit, точное состояние после rollback, backup restore, переходы breaker, отсутствие retry для 4xx и расчёты reliability. Успешно пройдены 17 тестов. Compose-конфигурации проверены; контейнеры не запускались, поскольку Docker engine был недоступен [4].

9 Методика экспериментов

Среда выполнения: Windows-11-10.0.26300-SP0; Python 3.14.6. Начало набора измерений (UTC): 2026-10-05T09:41:17.045013+00:00. Нагрузка направлялась к реальным локальным HTTP-процессам. По три повторения каждого сценария и режима дают 54 изолированных запуска; порядок baseline/FT чередуется между повторениями.

В каждом запуске используется новая папка базы. Сначала записывается начальный платёж, затем четыре warm-up probes и внедряется fault. В crash/database/node выполняются семь probes при активном сбое; затем процессы перезапускаются, стенд ждёт liveness и 1.05 s cooldown breaker, после чего выполняет пять recovery probes. Между завершёнными запросами выдерживается 80 ms. В timeout выполняются пять probes при сбое и пять после recovery с ожиданием 1.1 s после ремонта.

Timeout создаётся задержкой сервера на 1400 ms. Node fault одновременно завершает payment1 и academic. Load включает 120 уникальных платежей по 100 тиын с concurrency 30; duplicate - двенадцать одновременных доставок одного ключа. Interruption срабатывает один раз между изменением баланса и вставкой в ledger. Lost response имитирует 503 после commit. Backup возвращает предыдущий checkpoint и проверяет точное совпадение состояния audit.

Показатель	Определение измерения
Detection time	От fault injection до первого downstream failure, storage error или rollback в logs. Все wall clocks относятся к одному хосту. Если сигнала нет, указывается n/a.
Service recovery	От injection до первой последующей полноценной успешной probe. При маскировании через failover это время подтверждения успеха, а не downtime.
Component restoration	От injection до отметки о завершении restart/reset в стенде. Восстановление компонента отделено от успешного ответа сервиса.
Request availability	Число полноценных успешных ответов / число измеренных запросов. Degraded-ответы academic с HTTP 200 считаются отдельно.
Consistency	initial_due - due = SUM(payment.amount); нет duplicate keys; начальный платёж сохранён. HTTP success в baseline может скрывать нарушение данных.
Recovered requests	Полноценные успешные запросы клиента, потребовавшие более одной попытки gateway. Ненаблюдаемые downstream retries не учитываются.

Для интервалов запросов и событий используется perf_counter; structured logs содержат UTC wall time для сопоставления обнаруженных сбоев. Time availability оценивается по probes, а не непрерывно. Concurrent load-запросы не задают интервалы downtime; probes до и после burst могут пропустить отказ внутри него. Поэтому временные метрики интерпретируются только для crash, database, timeout и node.

10 Результаты и сравнение

Сценарий	Baseline full	FT full	B consistent	FT consistent
Application crash	27/48	48/48	3/3	3/3
Database outage	27/48	27/48	3/3	3/3
Academic timeout	33/48	33/48	3/3	3/3
Node A loss	30/54	51/54	3/3	3/3
Interrupted payment	27/30	30/30	0/3	3/3
Concurrent load	384/384	384/384	3/3	3/3
Duplicate delivery	60/60	60/60	0/3	3/3
Checkpoint restore	30/30	30/30	3/3	3/3
Lost acknowledgement	27/30	30/30	0/3	3/3

Таблица 1. Объединены три повторения. Full - полноценные успешные ответы / все запросы. Consistent - число повторений, выполнивших оба бизнес-инварианта. Дополнительные проверки student/payment в node и timeout включены в итоги. Backup представляет отдельную проверку recovery; у baseline нет функции восстановления checkpoint.

В смешанной нагрузке baseline дала 87 неуспешных полноценных ответов из 732, FT - 39 из 732. В FT 18 запросов восстановились благодаря retry. Итоги описывают только этот набор сценариев и не являются общим availability SLO.

Replica failover устранил ошибки получения списка платежей при application crash. При node loss в FT academic endpoint всё равно возвращал degraded-ответы, учтённые как потеря полноценной услуги. При storage outage обе версии теряли полноценные ответы зависимых сервисов: storage replication отсутствует. В baseline сценарии duplicate и lost reply могли возвращать HTTP success при нарушении уникальности платежа, поэтому availability и integrity оцениваются отдельно.

Режим	Среднее requests/s	Диапазон повторений	Среднее case p95, ms
baseline	114.0	65.8 - 150.6	384.4
ft	164.4	105.4 - 205.5	225.5

Таблица 2. Throughput рассчитан по 120 завершённым запросам burst; все они успешны в представленном наборе. p95 - среднее значений nearest-rank latency percentile отдельных запусков, включая probes до и после нагрузки. Трёх повторений недостаточно для статистически устойчивого вывода о преимуществе производительности.

Расчёты reliability к разделу 10

T - интервал от начала первого запроса до итогового audit; D - суммарная длительность объединённых интервалов downtime по probes; $U=T-D$; F - число эпизодов отказа; C - число завершённых recovery. Эпизод начинается при завершении первой неуспешной полноценной probe и заканчивается при завершении следующей полноценной успешной. Незавершённый в конце наблюдения эпизод ограничивается моментом T.

Оценка $MTTF = U/F$; $MTTR$ = сумма завершённых интервалов downtime / C; оценка $MTBF = MTTF + MTTR$; time availability = U/T ; observed failure rate = F/U . Request availability = полноценные успехи / N; retry recovery fraction = успешные запросы с retry / все запросы с retry. Если знаменателя нет, ставится n/a. Оценки $MTTF/MTBF$ для repairable cycle используют ограниченное окно наблюдения и описывают внедрённые отказы, а не естественный срок безотказной работы.

Сценарий	Режим	MTTF, s	MTTR, s	MTBF, s	Time A	Rate /s
crash	baseline	2.004	8.708	10.713	18.7%	0.499
crash	ft	n/a	n/a	n/a	100.0%	0.000
database	baseline	1.811	7.537	9.348	19.4%	0.552
database	ft	2.086	3.014	5.100	40.9%	0.479
timeout	baseline	1.897	5.650	7.547	25.1%	0.527
timeout	ft	1.662	1.607	3.268	50.8%	0.602
node	baseline	2.182	9.945	12.127	18.0%	0.458
node	ft	n/a	n/a	n/a	100.0%	0.000

Таблица 3. Объединены время наблюдения и recovery трёх повторений. Для FT node time availability относится к payment probes. Отдельный degraded transcript учтён в таблице 1, но не как payment outage. Поэтому в FT crash и node нет наблюдаемого payment outage, по которому можно оценить MTTF или MTTR.

Сценарий	B detect, ms	FT detect, ms	B confirm, ms	FT confirm, ms
crash	1139.5	544.0	9849.4	145.6
database	934.7	437.5	8474.9	4234.2
timeout	1013.7	317.1	6665.6	2402.2
node	1293.4	685.1	11240.2	293.3
interruption	14.1	10.6	41.6	82.8
response_loss	19.1	13.7	41.0	83.0

Таблица 4. Средние интервалы от injection до detection и первого подтверждённого success. В них входит расписание нагрузки. Более долгие timeout probes в baseline могут задерживать запланированный стендом repair. Поэтому разницу recovery нельзя целиком объяснять более быстрым ремонтом инфраструктуры: на неё также влияет методика запуска.

11 Обсуждение и ограничения

Для выбранных FT-сценариев учебные цели availability и integrity выполнены: при каждом single-replica crash сохранены полноценные payment-ответы, все FT audits согласованы, после восстановления возобновились обычные probes. Данные не доказывают непрерывную работу при потере базы или всего хоста. Реальное развёртывание потребовало бы redundant gateway, database replication, отдельных хостов и проверенной политики backup recovery.

Load test ограничен по объёму; база сериализует платежи с помощью process lock. Поэтому replication в основном защищает от отказа процесса, а не масштабирует запись. В baseline намеренно ослаблены границы транзакции. Сравнение имеет учебную цель и не является сравнением с корректно спроектированной production payment system.

Параметры timeout и retry выбраны для локального демо. HTTPX timeout ограничивает отдельные сетевые фазы, а не строгий end-to-end deadline [3]. Timeout между сервисом и базой составляет 800 ms. Retry выполняется только в gateway, чтобы избежать многократного умножения попыток. Для production нужны jitter, load shedding, общий deadline, ограниченный cache и tracing между хостами.

Состояние circuit breaker и transcript cache находятся в одном gateway-процессе. Health endpoints подтверждают liveness, но не readiness базы. Обнаружение отказа пассивное: оно происходит при обращении к неисправному endpoint. Node simulation не воспроизводит сбой CPU, ядра, диска или сетевого коммутатора. Physical hardware redundancy, RAID и ECC не измерялись.

Доступны только три повторения на одном хосте, конечная нагрузка и искусственно внедрённые faults. Нет оснований заявлять confidence interval или естественную failure rate. Оценки по probes пропускают время до обнаружения первой ошибки и могут не заметить короткий outage между запросами. Отсутствие измерений внутри load burst не считается доказанной availability.

Restore удаляет платежи и ключи после checkpoint. Его следует выполнять в maintenance/quiescence window с последующим reconciliation. Backup в демо хранится на том же диске. Приложение предназначено для локального обучения, его бизнес-API не имеет пользовательской аутентификации и не должно быть публично доступно.

12 Заключение

Небольшая университетская система позволяет наглядно исследовать fault tolerance. Application replication защищает выбранный сервис при потере процесса; circuit breaker ограничивает повторные обращения к неисправной зависимости; rollback и idempotency сохраняют integrity платежей. Эксперименты показывают и границы защиты: общая база и хост остаются общей областью отказа. Исходные запросы, event logs и снимки audit делают выводы воспроизводимыми.

13 Источники

[1] SQLite. Atomic Commit in SQLite. Дата обращения: 5 октября 2026 года.

<https://www.sqlite.org/atomiccommit.html>

[2] SQLite. Online Backup API. Дата обращения: 5 октября 2026 года.

<https://www.sqlite.org/backup.html>

[3] HTTPX. Timeouts. Дата обращения: 5 октября 2026 года.

<https://www.python-httpx.org/advanced/timeouts/>

[4] Docker. Control startup and shutdown order in Compose. Дата обращения: 5 октября 2026 года.

<https://docs.docker.com/compose/how-tos/startup-order/>

[5] Microsoft Learn. Circuit Breaker pattern. Дата обращения: 5 октября 2026 года.

<https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker>

[6] FastAPI. Deployment concepts. Дата обращения: 5 октября 2026 года.

<https://fastapi.tiangolo.com/deployment/concepts/>

Первичная документация [1-6] описывает используемые механизмы и принципы развёртывания. Все измеренные значения получены из экспериментальных записей этого репозитория, а не из примеров в источниках.

14 Приложение с кодом конфигурацией logs и дополнительными результатами

Материал	Содержание
app/	Gateway, управление fault injection, бизнес-сервисы, recovery policy, база и dashboard.
scripts/	Launcher дочерних процессов, эксперименты, метрики, проверка данных и сборка отчётов.
tests/	17 автоматических unit tests и HTTP integration tests.
compose.yaml и baseline override	Топология контейнеров, liveness checks, реплики и отдельные bind-mounted данные.
results/final/	54 JSON-сценария, summary.csv, сведения о среде и JSON event logs сервисов.
docs/	Архитектура, checklist задания, инструкции для demo и защиты, готовые отчёты.

Воспроизведение: `python -m pytest -q; python -m scripts.experiments --trials 3; python -m scripts.verify_results results/final`.

Новые эксперименты создают отдельные папки. Английский отчёт собирается командой `python -m scripts.build_report --dataset results/final`; русский - `python -m scripts.build_report_ru --dataset results/final`. Зависимости указаны в `requirements-report.txt`; русская версия использует Arial TTF с поддержкой кириллицы.

В logs записаны request IDs, UTC timestamps, service instance, mode, action, status и latency. Transaction events содержат ключ платежа; поля fault, attempt и recovery добавляются к соответствующим событиям. Отсутствующие поля не заменяются вымышленными измерениями.